MESA COHORT C4 · AI AND ITS APPLICATIONS · FINAL EXAMINATION · PROBLEM 10 (HIGH DIFFICULTY)

VC Pulse: the complete study guide

VC Fundraising Pipeline Intelligence for Vikram: the problem, the options we weighed, what we built, how we checked it, and every viva question we could think of. Written for someone new to AI.

Live app	https://vc-pulse-lime.vercel.app
How it works page	https://vc-pulse-lime.vercel.app/brief
GitHub (public)	https://github.com/nawazalam4u/vc-pulse
Images to submit	Downloads/VC_Pulse_Components_Map.png · Downloads/VC_Pulse_Nine_Checks.png

1 · The problem, in plain words

For a 15-year-old: Vikram started a company that makes software for trucking and delivery businesses. To grow, he needs money from investors called VCs (venture capitalists). He has a list of 50 of them. Writing to an investor is like asking someone to join your team: if you ask when they're already thinking about your topic (they just wrote about trucks, or just invested in a delivery company), they're far more likely to say yes. If you ask someone who has been silent for three months, you're probably wasting your time.

Vikram's list tells him **who** the investors are and **when he last spoke to them**. It doesn't tell him **who is "warm" right now**. Finding that out means searching the internet for 50 people, twice a week. Nobody has time for that, so he guesses.

Our job: build a robot that does that searching twice a week, reads everything, decides who is warm and why (with proof he can click), tells him who is quiet, and writes the answer into his list. The robot must **never** scribble over his own private notes.

What exactly has to happen (from the brief)

1. Twice a week, before Vikram starts work, check all 50 investors.
2. Use **observable** signals from the **last 30 days**: something they **wrote**, **said** in an interview, **invested** in, or **appeared** at.
3. Say who is worth contacting **this week, and why**. Not a hunch.
4. Also say who is **quiet**, so he knows where **not** to spend energy.
5. Update his table each run, so it is **already up to date** when he opens it.
6. Update **only** 4 columns (last signal date, signal summary, follow-up score, recommended action). His contact history and notes must stay **untouched**.
7. Exam rules: live external data; working with data at submission; **Gemini Flash** only (no Claude API); deployed on **Vercel**; submit URL + GitHub + Components Map.

Requirement → how VC Pulse meets it

Brief says	VC Pulse

Twice a week, before work	Vercel Cron, Mon + Thu 07:00 IST (within the hour on the free plan), plus a Refresh button
Observable signal, past 30 days	Every signal has a source link + the source's own date; older items are dropped before the AI sees them
Wrote / said / invested / appeared	Gemini labels each item with exactly these kinds (plus "in the news")
Who to reach out to this week, and why	Score 0-100 + action (REACH OUT / NUDGE / WARM / WAIT / LOW / PARK) + one-line reason + clickable evidence + email draft
Who is quiet	PARK, worded by how long they've been quiet (1-year lookback: "quiet for 3+ months since 25 May") and by the relationship ("waiting 60 days too: one light check-in next month, or move on")
Table reflects current state each run	Each run rewrites the 4 signal columns for every row from that run's evidence; the dashboard *is* that table
Never overwrite notes / contact history	Postgres column permissions: the engine login physically can't write them; fingerprint proof per run; live "attack" demo
"Supabase table"	Neon Postgres (Supabase is also Postgres; same SQL and permissions; switch = 2 connection strings). Reason: the free Supabase slots are used by earlier portfolio apps

2 · Possible solutions we considered (and why we chose this one)

Option	How it would work	Why we didn't choose it / what would go wrong
A. Google Alerts + spreadsheet	50 alerts emailed to Vikram; he reads them and updates the sheet	Still manual; no ranking, no "why", no relationship timing; 50 alerts flood the inbox; nothing updates the table. Doesn't meet "already up to date".
B. No-code automation (Zapier/Make)	RSS → AI step → Google Sheet row	Fast to start, but row-update steps typically rewrite whole rows, which risks overwriting notes (the critical rule). Hard to enforce "only 4 columns", to test, to explain the score, or to add guardrails. Paid per task.
C. Scrape LinkedIn / X	Pull each VC's posts directly	Richest signal, but against their terms; accounts get banned; brittle (pages change). Ethically and legally weak for a graded project.
D. Paid data (Crunchbase / PitchBook APIs)	Structured deals and people data	Expensive, needs approval; covers deals but not "wrote / said / appeared"; not "live" in the way news is.
E. "Ask the AI" only (Gemini + Google Search for each VC)	One prompt per VC: "what did they do in 30 days?"	Simplest, but (1) Google Search grounding isn't in the free Gemini plan, (2) the AI may invent dates or links (hallucination), (3) scores would change run to run and be unexplainable. Hard to defend in a viva.
F. Autonomous AI agent (LangChain-style)	An agent browses, decides and writes to the database	Over-engineered; unpredictable; slow; an agent with write access could touch the notes. The task is a fixed pipeline, not an open-ended one.
✔ G. Our hybrid pipeline	Official public feeds → Gemini Flash *judges* each real item (strict JSON) → code *checks* the AI and *calculates* the score → database permissions guard the notes	Live, free, legal; every claim has a link; same evidence = same score; the critical rule is enforced by Postgres, not by trust; testable (22 rule tests, 26 permission tests, a 57-item accuracy evaluation); cheap enough to run forever.

One-line answer for the viva: "AI is good at reading and judging meaning, bad at being consistent and honest about facts. So we let Gemini do only the judging, on real evidence we fetched, and let code and the database handle facts, maths and safety."

3 · The Nine Checks

VC Pulse · The Nine Checks

VC fundraising pipeline intelligence · MESA C4 final · Nawaz Alam · vc-pulse-lime.vercel.app

01 KILL SWITCH — PASS
Problem real
Timing decides if an investor reads your email. 50 VCs × 5 places × twice a week ≈ 500 lookups a week, so founders guess.

02 KILL SWITCH — PASS
Workflow repeated
Every Monday and Thursday for the whole 3-6 month raise: the same research loop ~50 times.

03 KILL SWITCH — PASS
Input available
Public, official feeds: Google News RSS, VC blog RSS, Apple Podcasts API. No LinkedIn/X scraping (terms, bans).

04 SIZING — PASS
Output valuable
Ranked weekly board, one clear action per VC, clickable evidence, and an email draft that cites the signal.

05 SIZING — PASS
Impact measurable
Research hours → minutes; % of outreach with a real hook; reply rate of timed emails; AI accuracy measured (100% precision on 57 items).

06 BOUNDARY — PASS
Failure risk OK
Every signal has a link + date; evidence must name the investor; code checks the AI; outages keep last verdict. Worst case: one mistimed email.

07 BOUNDARY — PASS
Judgment protected
AI recommends, Vikram decides and sends. Nothing is emailed automatically; his notes are locked by database permissions.

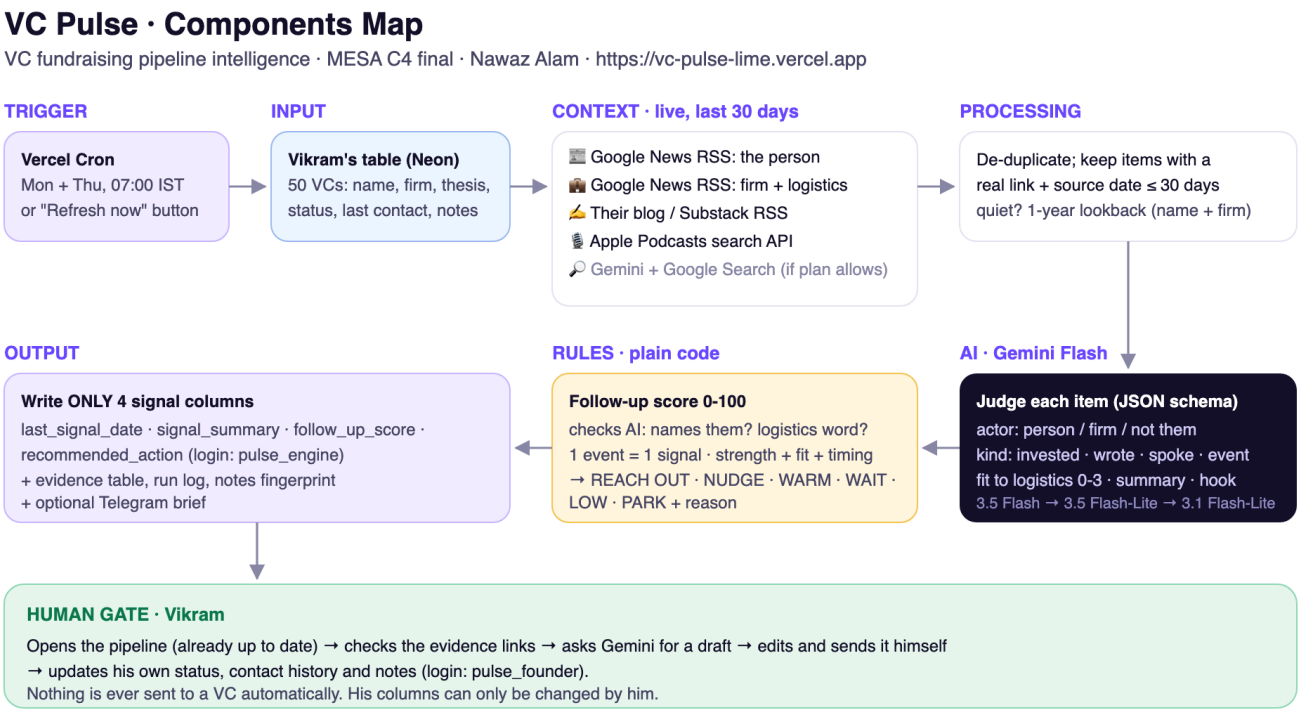
08 SIZING — PASS
ROI worth it
≈ ₹0 to run (free tiers; ~5 AI calls per run) vs 4-5 founder-hours saved every week of the raise.

09 BOUNDARY — PASS
Owner clear
Vikram owns relationships + his columns; the engine login owns 4 signal columns; the builder owns sources, prompt and rules (public repo).

The Cut (deliberately NOT automated): sending emails (Vikram sends) · writing his notes or contact history (blocked by the database) · scraping LinkedIn/X (terms of service) · letting the AI set the score (AI classifies, code calculates).

Kill switches (01-03): if any fails, don't build. Sizing (04, 05, 08): is it worth it? Boundary (06, 07, 09): where must humans stay in charge? **The Cut** is what we deliberately left to Vikram.

4 · Components Map and how one run works



1. **Trigger:** Vercel Cron (Mon + Thu 01:30 UTC = 07:00 IST) or the Refresh button. Only one run at a time (a unique index in the database); the scheduled run always wins over a half-finished manual one.
2. **Input:** the 50 rows of Vikram's table in Neon.
3. **Context (live):** for each investor, in parallel: Google News for the person; Google News for the firm + logistics words (deals); the blog/Substack feed on file; Apple Podcasts. Only items with a real http(s) link and a source date in the last 30 days. Each source keeps its own share, so a strong older deal isn't crowded out.
4. **AI (Gemini Flash):** 10 investors per call, strict JSON schema, temperature 0.1. For each item: actor (person / firm / not them), kind, logistics fit 0-3, event id, summary; per investor: overall summary and email hook (with the item it's based on).
5. **Code checks the AI:** the text must name the investor (or, for a deal, name the firm or be explicitly about logistics); "3/3 logistics" needs a logistics word in the text; blog posts count for the partner only if they're the author; one real-world event counts once; the hook must come from a kept item.
6. **Rules (code):** score and action from fixed weights plus Vikram's status and last-contact date (read-only). For quiet investors: a 1-year lookback of news naming them with their firm.
7. **Output:** one targeted UPDATE of the 4 signal columns per investor through the restricted `pulse_engine` login. Evidence, score changes and the run log go to separate tables. A fingerprint of Vikram's columns is taken before and after.
8. **Human gate:** Vikram reviews, clicks the evidence, asks Gemini for a draft, edits and sends it himself, and updates his own columns.

Worked example: why 8VC's Jake Medwell scored 59 (REACH OUT)

Step	Value
Evidence	"Gallatin AI raises $50M Series A to digitize military logistics" (SiliconANGLE, 9 Oct; found by searching "8VC" + logistics words; 6 more outlets = same event, counted once)

Gemini's judgement	actor = firm, kind = invested, logistics fit = 3, event = gallatin-series-a
Code checks	deal + logistics word in the headline ✓ · logistics word present ✓
Signal points	40 (invested) × 1.0 (logistics) × 0.85 (firm, not the partner personally) × 1.0 (≤ 7 days old) = **34**
Thesis fit	thesis mentions logistics / transport → **+15**
Timing	not contacted yet + fresh signal → **+10**
Total → action	34 + 15 + 10 = **59** ≥ 55 → **REACH OUT** with the hook "Saw 8VC lead Gallatin AI's $50M Series A…"

All the actions

Action	When	Meaning for Vikram
🔥 REACH OUT	score ≥ 55, not contacted yet	Write this week, using the hook
🔥 NUDGE	score ≥ 55, waiting 14+ days for a reply	A real reason for a polite follow-up
🌤️ WARM	35-54	Active; worth it if you have time
⏳ WAIT	followed up less than 14 days ago	Don't pester; the hook is saved for when the wait is over
· LOW	weak, old or off-topic activity	Busy, but not with your space
💤 PARK	no qualifying signal in 30 days	Quiet; worded by how long (3+ months?) and by the relationship

5 · How do we know it works? (accuracy, tests, runs)

100%
precision: no false claims about who did what (57 hand-labelled items)

89%
recall: found 24 of 27 real signals; misses were headlines that didn't name the investor

48/48
automated tests pass (22 scoring rules + 26 database permissions)

0
errors in the final live runs; notes fingerprint identical before and after every run

How the accuracy test works: we took 57 real items from the live pipeline and labelled each by hand (is it really about this investor or their firm? is it about logistics?). Then we ran them through Gemini plus our code checks (`tests/eval_judge.py`) and compared. The topic label was 100% right on the correct items. This was measured on the fallback model (Gemini 3.5 Flash-Lite), so it's the worst case.

Why precision over recall: a missed signal costs Vikram a little; a wrong one ("congrats on your post" when a colleague wrote it) could embarrass him in front of an investor. So the rules are strict on purpose.

Live runs: all 50 investors in 40-60 seconds; ~400 public items checked per run; ~60-80 kept as real signals; both the Refresh path and the scheduled cron path tested on the real server.

6 · The professor's audit: what we found and fixed

Before submitting, we ran a deliberately harsh review: an independent code reviewer, a data audit of every recommendation, and live testing of every button. Each finding was fixed and re-tested.

Severity	Finding	Fix
High	If a news source failed, the investor was marked "quiet" and old evidence deleted	Any news-source failure → keep last verdict, log it, mark run "partial"
High	A malicious feed link (`javascript:`) could run code in Vikram's browser	Only http(s) links stored and shown; strict Content-Security-Policy; key sent only in a header
High	Anyone could keep a run going (spend AI quota) or block the scheduled run	HMAC run tickets; scheduled run takes over stale manual runs; time budget
High	Two runs could start at the same instant and fight	Unique index: at most one "running" row, enforced by Postgres
High	Firm-wide blogs (a16z) credited colleagues' posts to our partner ("your post")	Read each post's author; otherwise firm-level only; hooks never say "your" for firm acts
Medium	"Lumio" consumer deal credited to Stellaris; "Eclipse" car reviews; "11th October 2026" pages	Evidence must name the investor or firm (or be an explicit logistics deal)
Medium	"Quick delivery" deals counted as logistics for firms not named	Strict logistics word list for firm deals
Medium	7 articles about one deal counted 7 times	Gemini event ids; each event counted once ("same event, other outlets")
Medium	AI rated a real-estate interview as "logistics"	Code caps "3/3" without a logistics word in the text

Medium	Same 8VC deal "yes" one run, "no" the next	Told Gemini how each item was found (firm-name search) → stable
Medium	"Followed up 9 days ago" could still say "NUDGE this week"; negative days for future dates	WAIT for 14 days after a follow-up; dates clamped
Medium	"Quiet: nothing in 30 days. Last signal 7 Oct" contradiction	"Last seen" uses only history older than the window + this run
Medium	Quiet actions ignored the relationship; couldn't tell 1 month from 3 months quiet	Relationship-aware PARK wording + 1-year name+firm lookback
Medium	Email hook could cite an item the code had rejected	Hook must point to a kept item; otherwise a factual hook from its title
Medium	Scheduled run had no time limit (could time out half-way)	Stops starting new work after 200 s of the 300 s limit
Medium	CSV import: "Waiting" status lost, dd/mm dates failed, Excel BOM broke headers	Status synonyms, Indian date formats, BOM stripped; export blocks Excel formulas
Medium	Free Gemini 3.5 Flash = 20 calls/day; drafts and probes could burn it	10 investors per call, exhausted models skipped, drafts use Lite, 30/hour visitor cap, probe editor-only
Low	Cards not usable by keyboard; drawer/modal lacked dialog roles; click race in drawer	tabindex, Enter/Space, ARIA roles, focus, latest-click-wins
Low	Fingerprint could miss a value moving into an empty column	NULL-safe fingerprint
Low	Podcast search matched common surnames (Reddy, Pai)	Exact full-name match
Low	Long lanes, no "add investor", poor link preview	"Show all" lanes, Add-investor form, preview image
Known	Notes visible in the public demo	Deliberate so reviewers can verify; editing needs the private key; production would add login

7 · Tools we used, and why

Tool	What it is (simple)	Its job here
Vercel	Puts websites and small server programs on the internet	Hosts the dashboard + Python API; Cron runs the refresh Mon/Thu
GitHub	An online "time machine" for code; public = portfolio	Stores code; each push auto-deploys
Neon (Postgres)	A cloud database	Vikram's table, evidence, runs, drafts; enforces column permissions
Gemini Flash (3.5 Flash → 3.5 Flash-Lite → 3.1 Flash-Lite)	Google's fast AI models (exam requirement)	Judges items, writes summaries, hooks and drafts
Google News RSS	Free public news feed for apps	Interviews, quotes, events, deals
Blog / Substack RSS	Feeds blogs publish for readers' apps	What investors wrote (author-checked)
Apple Podcasts API	Apple's public podcast search	Podcasts they spoke on
Neon CLI, Vercel CLI, gh	Command-line tools	Created the database, project, repo and deployments
Python · HTML/CSS/JS	Programming languages	Back end + tests · the dashboard (no frameworks, loads fast)

8 · Words you need to know

API	A way for programs to ask other programs for things over the internet (like a waiter taking your order to the kitchen).
API key / secret	A password for an API. Ours live in Vercel's hidden settings, never in GitHub.
Serverless function	Code that wakes up when called; we don't manage any computer.
Cron	A schedule for servers. "30 1 * * 1,4" = 01:30 UTC every Monday and Thursday.
RSS	A public list of a site's newest articles, made for apps.
LLM	Large language model: AI that reads and writes language (Gemini).
Prompt / system instruction	The AI's job description and rules.
Structured output (JSON schema)	Forcing the AI to answer in an exact format so code can read it.
Temperature	Creativity dial: 0.1 for judging (consistent), 0.6 for email drafts.
Hallucination	AI stating something false. Prevented: the AI only labels real items with real links.
Grounding	Making AI answer from real sources (our fetched items).
Precision / recall	Precision: of what it flagged, how much was right. Recall: of what was really there, how much it found.

Least privilege	Each part gets only the permissions it needs.
Transaction	Several database changes that all succeed or all fail.
Hash / fingerprint	A short code computed from data; any change changes it. Proves notes didn't change.
HMAC token	A tamper-proof ticket made with a secret; proves you started a run.
CSP	Content-Security-Policy: browser rule that only our own scripts may run.
Rate limit / quota	How many requests a service allows (free Gemini 3.5 Flash: 20 per day).
Fallback	Plan B: if one model is busy, use the next.
Deterministic	Same input → same output (our scoring code).
Prompt injection	Text inside data trying to give the AI orders. Limited here: the AI can only fill a fixed schema, and code checks its answers.

---

9 · Problems we hit while building (good viva stories)

- **Gemini Google Search = "quota exceeded".** Not in the free plan → built on free official feeds; search switches on automatically if billing is enabled.
- **3.5 Flash took 80 s for 5 investors.** "Thinking level: low" → 25 s with the same quality.
- **20 calls/day limit.** Batched 10 investors per call; skip exhausted models; Lite for drafts.
- **Vercel couldn't find helper files.** Added the folder to Python's path in every endpoint (test on the real server!).
- **Accuracy problems found by auditing real outputs** (wrong firm, car reviews, colleague's posts): fixed with code checks, then measured (100% precision).

10 · The 3-minute demo for the professor

1. **vc-pulse-lime.vercel.app:** "This is Vikram's week. 🔥 who to contact, 🌤️ warm, ⏳ wait, 💤 quiet. The strip shows how it works."
2. Click the top card: "Here's the proof: a real deal from this week, the link, the date, and the score maths line by line." Press **Draft the email**: "Draft only; Vikram sends."
3. Open a PARK card: "Quiet for 3+ months, and he's been waiting 34 days for a reply, so don't chase."
4. **Pipeline table:** "Blue columns are Vikram's, purple are the engine's, the same table that lives in the database."
5. **Notes guard → Attack my own database:** "Postgres refuses: permission denied. Fingerprints match on every run."
6. **Run history:** "Runs automatically Monday and Thursday at 7 AM; this one was the scheduled path." Optionally press Refresh (~50 s).
7. **How it works** page: brief, Nine Checks, The Cut, accuracy, security, Components Map.

11 · Viva questions and answers

1. The problem in one sentence?

Tell a fundraising founder which of his 50 investors are worth contacting this week, with clickable proof from the last 30 days, and keep his pipeline table updated twice a week without ever touching his own notes.

2. Where exactly is the AI, and why is it needed?

Gemini Flash reads each headline and decides whether it's really this investor, what they did, and whether it's about logistics, then writes the summary, hook and drafts. Keyword rules can't tell "Peak XV led a round" from "a company Peak XV once backed raised from others".

3. Why not let the AI decide the score?

Scores must be consistent and explainable. AI judges meaning; code does the arithmetic. Same evidence → same score, shown line by line.

4. How do you prevent hallucinations?

The AI only labels items we fetched (real URL + source date); strict JSON; temperature 0.1; code checks (evidence names them, logistics word, author check, one event once, hook from a kept item); rejected items stay visible.

5. How accurate is it?

On 57 hand-labelled live items: 100% precision, 89% recall, 95% overall; topic label 100%. Measured on the fallback Lite model (worst case).

6. Why prefer precision over recall?

A missed signal costs little; a false "congrats on your post" damages trust with an investor.

7. Prove the notes can't be overwritten.

Column-level Postgres permissions: `pulse_engine` can UPDATE only the 4 signal columns. Live "attack" shows "permission denied"; per-run MD5 fingerprints match; updates target 4 columns by id, never whole rows.

8. Why two database logins?

Separation of duties: the robot can't touch notes; the edit screen can't fake a score. The owner login isn't on the server at all.

9. What is the live external data?

Google News RSS (person; firm + logistics), the investors' blog/Substack RSS, the Apple Podcasts API, all fetched at run time.

10. Why not LinkedIn / X?

Scraping breaks their terms and gets accounts banned. A real product would use official APIs with permission.

11. Why Neon, not Supabase?

Both are Postgres; schema and permissions are plain SQL and run unchanged on Supabase; the free Supabase slots are used by earlier projects. Switching = 2 connection strings.

12. How is "twice a week" implemented?

Vercel Cron calls `/api/cron` with a secret at 01:30 UTC Mon/Thu. It processes 50 investors in parallel within a 200-second budget.

13. What if Gemini or a news source is down?

Gemini falls back across three Flash models. If the AI or a news source still fails for an investor, they keep last run's verdict and the run is marked "partial". We never wrongly call someone "quiet".

14. How do you know someone is quiet for 3 months, not just 30 days?

For investors with no 30-day signal, a 1-year news search for name + firm together (avoids namesakes) gives the last mention date.

15. How does the relationship affect the advice?

Followed up under 14 days ago → WAIT (hook saved). Waiting 14+ days + fresh signal → NUDGE. Waiting 45+ days and quiet → "one light check-in next month, or move on".

16. Same person, different name problem?

Evidence must name the investor or firm; lookback uses name + firm; podcasts need the exact full name; Gemini's actor check rejects namesakes (e.g. a different Neeraj Arora).

17. What stops two runs colliding?

A unique index allows only one "running" run; the scheduled run supersedes a stale manual run; batch calls need an HMAC run ticket.

18. What does a run cost?

About ₹0: ~5 Gemini calls (free tier), free feeds, Vercel Hobby, Neon free.

19. How did you choose the weights?

Expert heuristics from the brief: a deal in your space is the strongest signal, then writing, speaking, events; freshness decays over 30 days; timing respects follow-up etiquette. They're in one place in code and the next step is learning them from which emails got replies.

20. What does the JSON schema do?

It forces Gemini to return exact fields with allowed values (e.g. kind ∈ invested/wrote/said/appeared/mentioned/other), so code can rely on it.

21. Why Gemini 3.5 Flash with "thinking: low"?

Careful enough for judging, 3× faster than default thinking; Lite models as fallback.

22. Security?

Secrets only in Vercel env; least-privilege DB logins; HMAC run tickets; CSP + http(s)-only links; editor key in a header; rate limits on public AI actions.

23. Is anything fake?

Investors, firms, theses and every signal are real and live. Vikram, "Haulr" and his notes are illustrative because he is a case character.

24. Why not an AI agent?

The task is a fixed pipeline; an agent adds unpredictability and would need write access near the notes.

25. How did you test?

22 scoring-rule tests (offline), 26 live permission tests, a 57-item accuracy evaluation, end-to-end runs on both run paths, and a harsh audit with every finding fixed.

26. What is the human's role?

Vikram decides whom to contact, edits and sends every email, and owns his notes. The tool only recommends.

27. Limitations?

Headline-level evidence (no full-article reading); public activity only; event coverage limited without paid search; weights not yet learned from outcomes.

28. What would make it a product?

Login and per-founder data (row-level security), Gmail integration to log replies (with consent), Slack/Telegram brief, learning weights from replies, paid search grounding for events.

29. How does Vikram add investors?

An "Add investor" form or CSV import (via his private link), using the founder login, which can't touch signal columns.

30. What did you learn?

Use AI for judgement on real evidence, box it in with a schema and context, verify it with code, measure it, and let the database enforce the rules that matter most.

---

12 · Submission checklist

- □ Working public URL: **https://vc-pulse-lime.vercel.app**
- □ GitHub: **https://github.com/nawazalam4u/vc-pulse**
- □ Components Map: **VC_Pulse_Components_Map.png** (Downloads) · Nine Checks: **VC_Pulse_Nine_Checks.png**
- □ For the demo: Vikram's private editor link (in the chat, not in this PDF; don't share publicly)